

Contents

1. Contest	1
2. Data Structures	2
3. Graph	7
4. Number Theory	23
5. Strings	29
6. Additional	31

1. Contest

1.1. rng.h

```
mt19937_64
rng(chrono::steady_clock::now().time_since_epoch().count());
```

```
int random(int a, int b) {
    if (a > b)
        return 0;
    return a + rng() % (b - a + 1);
}
double random_double(double a, double b) {
    return a + (b - a) * (rng() / (double)rng.max());
}
```

1.2. template.h

```
#include <bits/stdc++.h>

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;
typedef tree<int, null_type, less_equal<int>, rb_tree_tag,
            tree_order_statistics_node_update>
    ordered_multiset;
typedef tree<int, null_type, less<int>, rb_tree_tag,
            tree_order_statistics_node_update>
```

```
    ordered_set;
// find_by_order returns iterator to kth largest (0-indexed)
// order_of_key returns number of elements strictly less than
// given value -->
// basically index (0-indexed) for multiset, to erase use
// upper_bound.
// Upper_bound lower_bound exchange their roles
#define ONLINE_JUDGE
#ifndef ONLINE_JUDGE
#define db(x) cerr << #x << " == " << x << endl
#define dbs(x) cerr << x << endl
#else
#define db(x) ((void)0)
#define dbs(x) ((void)0)
#endif

#define int long long
#define fast()
\
    ios_base::sync_with_stdio(0);
\
    cin.tie(NULL);
\
    cout.tie(NULL);
#define fr(i, a, b) for (int i = (a); i < (int)(b); ++i)
#define pb push_back
#define prDouble(x) cout << fixed << setprecision(10) << x
int M = 1e9 + 7;
#define all(x) x.begin(), x.end()
#define allr(x) x.rbegin(), x.rend()
#define sz(x) (int)x.size()
void solve() {}
signed main() {
    fast();
    int t = 1;
    cin >> t;
```

```

while (t--) {
    solve();
}
return 0;
}

```

2. Data Structures

2.1. DSU.h

```

struct DSU {
    int n;
    vi parent;
    vi size;
    DSU(int _n) : n(_n), parent(n), size(n, 1) {
        iota(parent.begin(), parent.end(), 0);
    }
    int find_set(int x) {
        if (parent[x] == x)
            return x;
        return parent[x] = find_set(parent[x]);
    }
    int get_size(int x) {
        return size[find_set(x)];
    } // returns size of component of x
    void union_sets(int x, int y) {
        x = find_set(x);
        y = find_set(y);
        if (x == y)
            return;
        if (size[x] > size[y]) {
            parent[y] = x;
            size[x] += size[y];
        } else {
            parent[x] = y;
            size[y] += size[x];
        }
    }
}

```

```

}
};

```

2.2. LazySegTree.h

```

template <typename T, typename U> struct seg_tree_lazy {
    int S, H;
    T zero;
    vector<T> value;
    U noop;
    vector<bool> dirty;
    vector<U> prop;
    seg_tree_lazy(int _S, T _zero = T(), U _noop = U()) {
        zero = _zero, noop = _noop;
        for (S = 1, H = 1; S < _S;)
            S *= 2, H++;
        value.resize(2 * S, zero);
        dirty.resize(2 * S, false);
        prop.resize(2 * S, noop);
    }
    void set_leaves(vector<T> &leaves) {
        copy(leaves.begin(), leaves.end(), value.begin() + S);
        for (int i = S - 1; i > 0; i--)
            value[i] = value[2 * i] + value[2 * i + 1];
    }
    void apply(int i, U &update) {
        value[i] = update(value[i]);
        if (i < S) {
            prop[i] = prop[i] + update;
            dirty[i] = true;
        }
    }
    void rebuild(int i) {
        for (int l = i / 2; l; l /= 2) {
            T combined = value[2 * l] + value[2 * l + 1];
            value[l] = prop[l](combined);
        }
    }
}

```

```

}
void propagate(int i) {
    for (int h = H; h > 0; h--) {
        int l = i >> h;
        if (dirty[l]) {
            apply(2 * l, prop[l]);
            apply(2 * l + 1, prop[l]);
            prop[l] = noop;
            dirty[l] = false;
        }
    }
}
void upd(int i, int j, U update) {
    i += S, j += S;
    propagate(i), propagate(j);
    for (int l = i, r = j; l <= r; l /= 2, r /= 2) {
        if ((l & 1) == 1)
            apply(l++, update);
        if ((r & 1) == 0)
            apply(r--, update);
    }
    rebuild(i), rebuild(j);
}
T query(int i, int j) {
    i += S, j += S;
    propagate(i), propagate(j);
    T res_left = zero, res_right = zero;
    for (; i <= j; i /= 2, j /= 2) {
        if ((i & 1) == 1)
            res_left = res_left + value[i++];
        if ((j & 1) == 0)
            res_right = value[j--] + res_right;
    }
    return res_left + res_right;
}
};

```

```

struct node {
    int sum, width;
    node operator+(const node &n) {
        // change 1
        return {sum + n.sum, width + n.width};
    }
};
struct update {
    bool type; // 0 for add, 1 for reset
    int value;
    node operator()(const node &n) {
        // change 2
        if (type)
            return {n.width * value, n.width};
        else
            return {n.sum + n.width * value, n.width};
    }
    update operator+(const update &u) {
        // change 3
        if (u.type)
            return u;
        return {type, value + u.value};
    }
};
// Example Initialization
// seg_tree_lazy<node, update> lst(size, {0, 1}, {0, 0});
// lst.set_leaves(leaves);
// lst.upd(l, r, {0, value});
// auto result = lst.query(l, r);

```

2.3. Mos.h

```

int BLOCK = DO_NOT_FORGET_TO_CHANGE_THIS;
struct Query {
    int l, r, id;
    Query(int _l, int _r, int _id) : l(_l), r(_r), id(_id) {}
    bool operator<(Query &o) {

```

```

    int mblock = l / BLOCK, oblock = o.l / BLOCK;
    return (mblock < oblock) or
           (mblock == oblock and mblock % 2 == 0 and r < o.r)
or
           (mblock == oblock and mblock % 2 == 1 and r > o.r);
};
};
void solve() {
    vector<Query> queries;
    queries.reserve(q);
    for (int i = 0; i < q; i++) {
        int l, r;
        cin >> l >> r;
        l--, r--;
        queries.emplace_back(l, r, i);
    }
    sort(all(queries));
    int ans = 0;
    auto add = [&](int v) {};
    auto rem = [&](int v) {};
    vector<int> out(q); // Change out type if necessary
    int cur_l = 0, cur_r = -1;
    for (auto &[l, r, id] : queries) {
        while (cur_l > l)
            add(--cur_l);
        while (cur_l < l)
            rem(cur_l++);
        while (cur_r < r)
            add(++cur_r);
        while (cur_r > r)
            rem(cur_r--);
        out[id] = ans;
    }
}

```

2.4. RMQ.h

```

template <class T> struct RMQ {
    vector<vector<T>> jmp;
    RMQ(const vector<T> &V) : jmp(1, V) {
        for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k) {
            jmp.emplace_back(sz(V) - pw * 2 + 1);
            for (int j = 0; j < sz(jmp[k]); j++)
                jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
        }
    }
    T query(int a, int b) {
        assert(a <= b); // tie(a, b) = minimax(a, b)
        int dep = 63 - __builtin_clzll(b - a + 1);
        return min(jmp[dep][a], jmp[dep][b - (1 << dep) + 1]);
    }
};

```

2.5. SegTree.h

```

struct segtree {
    typedef int T;
    // for max segtree, set unit = INT_MIN and f(a,b) = max(a,b)
    static constexpr T unit = 0; // identity for sum
    T f(T a, T b) { return a + b; } // (any associative fn)
    vector<T> s;
    int n;
    segtree(int n = 0, T def = unit) : s(2 * n, def), n(n) {}
    void update(int pos, T val) {
        for (s[pos += n] = val; pos /= 2;)
            s[pos] = f(s[pos * 2], s[pos * 2 + 1]);
    }
    T query(int b, int e) { // query [b, e)
        T ra = unit, rb = unit;
        for (b += n, e += n; b < e; b /= 2, e /= 2) {
            if (b % 2)
                ra = f(ra, s[b++]);
            if (e % 2)

```

```

        rb = f(s[--e], rb);
    }
    return f(ra, rb);
}
};

```

2.6. Treap.h

/*A short self-balancing tree. It acts as a sequential container with log-time splits/joins, and is easy to augment with additional data.

Time: $\mathcal{O}(\log N)$ */

```

struct Node {
    Node *l = 0, *r = 0;
    int val, y, c = 1;
    Node(int val) : val(val), y(rand()) {}
    void recalc();
};

int cnt(Node *n) { return n ? n->c : 0; }
void Node::recalc() { c = cnt(l) + cnt(r) + 1; }

template <class F> void each(Node *n, F f) {
    if (n) {
        each(n->l, f);
        f(n->val);
        each(n->r, f);
    }
}

pair<Node *, Node *> split(Node *n, int k) {
    if (!n)
        return {};
    if (cnt(n->l) >= k) { // "n->val >= k" for lower_bound(k)
        auto pa = split(n->l, k);
        n->l = pa.second;
    }
}

```

```

        n->recalc();
        return {pa.first, n};
    } else {
        auto pa = split(n->r, k - cnt(n->l) - 1); // and just "k"
        n->r = pa.first;
        n->recalc();
        return {n, pa.second};
    }
}

```

```

Node *merge(Node *l, Node *r) {
    if (!l)
        return r;
    if (!r)
        return l;
    if (l->y > r->y) {
        l->r = merge(l->r, r);
        l->recalc();
        return l;
    } else {
        r->l = merge(l, r->l);
        r->recalc();
        return r;
    }
}

```

```

Node *ins(Node *t, Node *n, int pos) {
    auto [l, r] = split(t, pos);
    return merge(merge(l, n), r);
}

```

```

// Example application: move the range [l, r) to index k
void move(Node *&t, int l, int r, int k) {
    Node *a, *b, *c;
    tie(a, b) = split(t, l);
    tie(b, c) = split(b, r - l);
}

```

```

if (k <= l)
    t = merge(ins(a, b, k), c);
else
    t = merge(a, ins(c, b, k - r));
}

```

2.7. cht.h

```

struct Line {
    mutable i64 m, c, p;
    bool operator<(const Line &o) const { return m < o.m; }
    bool operator<(i64 x) const { return p < x; }
};

```

```

struct LineContainer : multiset<Line, less<>> {
    // (for doubles, use inf = 1/.0, div(a,b) = a/b)
    static const i64 inf = LONG_LONG_MAX;
    i64 div(i64 a, i64 b) { // floored division
        return a / b - ((a ^ b) < 0 && a % b);
    }
    bool isect(iterator x, iterator y) {
        if (y == end())
            return x->p = inf, 0;
        if (x->m == y->m)
            x->p = x->c > y->c ? inf : -inf;
        else
            x->p = div(y->c - x->c, x->m - y->m);
        return x->p >= y->p;
    }
    void add(i64 m, i64 c) {
        auto z = insert({m, c, 0}), y = z++, x = y;
        while (isect(y, z))
            z = erase(z);
        if (x != begin() && isect(--x, y))
            isect(x, y = erase(y));
        while ((y = x) != begin() && (--x)->p >= y->p)
            isect(x, erase(y));
    }
}

```

```

}
i64 query(i64 x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.m * x + l.c;
}
};

```

2.8. range_update_tree.h

```

template <typename T, typename F> struct RangeUpdateTree {
    int n;
    vector<T> tree;
    T identity;
    F merge;
    RangeUpdateTree(const vector<T> &arr, T id, F _m)
        : n((int)arr.size()), tree(2 * n), identity(id),
        merge(_m) {
        for (int i = 0; i < n; i++)
            tree[n + i] = arr[i];
        for (int i = n - 1; i >= 1; i--)
            tree[i] = merge(tree[2 * i], tree[2 * i + 1]);
    }
    void update(int l, int r, T value) {
        assert(l >= 0 && r < n && l <= r);
        for (l += n, r += n; l <= r; l >>= 1, r >>= 1) {
            if (l & 1)
                tree[l] = merge(value, tree[l]), l++;
            if (!(r & 1))
                tree[r] = merge(value, tree[r]), r--;
        }
        if (l == r)
            tree[l] = merge(value, tree[l]);
    }
    T query(int v) {
        T res = tree[v += n];
        for (; v > 1; v >>= 1)

```

```

        res = merge(res, tree[v >> 1]);
    return res;
}
};
// ex: RangeUpdateTree<int, decltype(join)> v(vi (n, 1e9), 1e9,
join); use auto
// func for join

```

3. Graph

3.1. 2sat.h

```

/*
ts.either(x, y);
ts.either(~x, ~y); these two do x xor y

ts.setValue(x, x); assert x is true

use ~x to denote not x

call ts.solve() to run the solver, returns if a solution exists
if exists: ts.values[i] contains the assignments
*/

struct TwoSat {
    int N;
    vector<vi> gr;
    vi values; // 0 = false, 1 = true

    TwoSat(int n = 0) : N(n), gr(2 * n) {}

    int addVar() { // (optional)
        gr.emplace_back();
        gr.emplace_back();
        return N++;
    }
}

```

```

void either(int f, int j) {
    f = max(2 * f, -1 - 2 * f);
    j = max(2 * j, -1 - 2 * j);
    gr[f].push_back(j ^ 1);
    gr[j].push_back(f ^ 1);
}

void setValue(int x) { either(x, x); }

void atMostOne(const vi &li) { // (optional)
    if (sz(li) <= 1)
        return;
    int cur = ~li[0];
    for (int i = 2; i < sz(li); i++) {
        int next = addVar();
        either(cur, ~li[i]);
        either(cur, next);
        either(~li[i], next);
        cur = ~next;
    }
    either(cur, ~li[1]);
}

vi val, comp, z;
int time = 0;
int dfs(int i) {
    int low = val[i] = ++time, x;
    z.push_back(i);
    for (int e : gr[i])
        if (!comp[e])
            low = min(low, val[e] ? dfs(e));
    if (low == val[i])
        do {
            x = z.back();
            z.pop_back();
            comp[x] = low;
            if (values[x >> 1] == -1)

```

```

        values[x >> 1] = x & 1;
    } while (x != i);
    return val[i] = low;
}

bool solve() {
    values.assign(N, -1);
    val.assign(2 * N, 0);
    comp = val;
    for (int i = 0; i < 2 * N; i++)
        if (!comp[i])
            dfs(i);
    for (int i = 0; i < N; i++)
        if (comp[2 * i] == comp[2 * i + 1])
            return 0;
    return 1;
}
};

```

3.2. Dinic.h

```

// Flow algorithm with complexity  $O(VE \log U)$  where  $U = \max |\text{cap}|$ .
//  $O(\min(E^{1/2}, V^{2/3})E)$  if  $U = 1$ ;  $O(\sqrt{V}E)$  for bipartite
// matching.
using ll = long long;
#define rep(i, j, k) for (int i = j; i < k; i++)
struct Dinic {
    struct Edge {
        int to, rev;
        ll c, oc;
        ll flow() { return max(oc - c, 0LL); } // if you need flows
    };
    vi lvl, ptr, q;
    vector<vector<Edge>> adj;
    Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}

```

```

    void addEdge(int a, int b, ll c, ll rcap = 0) {
        adj[a].push_back({b, sz(adj[b]), c, c});
        adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
    }
    ll dfs(int v, int t, ll f) {
        if (v == t || !f)
            return f;
        for (int &i = ptr[v]; i < sz(adj[v]); i++) {
            Edge &e = adj[v][i];
            if (lvl[e.to] == lvl[v] + 1)
                if (ll p = dfs(e.to, t, min(f, e.c))) {
                    e.c -= p, adj[e.to][e.rev].c += p;
                    return p;
                }
        }
        return 0;
    }
    ll calc(int s, int t) {
        ll flow = 0;
        q[0] = s;
        rep(L, 0, 31) do { // 'int L=30' maybe faster for random
data
            lvl = ptr = vi(sz(q));
            int qi = 0, qe = lvl[s] = 1;
            while (qi < qe && !lvl[t]) {
                int v = q[qi++];
                for (Edge e : adj[v])
                    if (!lvl[e.to] && e.c >> (30 - L))
                        q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
            }
            while (ll p = dfs(s, t, LLONG_MAX))
                flow += p;
        }
        while (lvl[t])
            ;
        return flow;
    }

```



```

    }
    bool leftOfMinCut(int a) { return lvl[a] != 0; }
};

```

3.3. DirectedMST.h

```

/**
 * Author: chilli, Takanori MAEHARA, Benq, Simon Lindholm
 * Date: 2019-05-10
 * License: CC0
 * Source:
 * https://github.com/spaghetti-source/algorithm/blob/master/
graph/arborescence.cc
 * and
 * https://github.com/bqi343/USACO/blob/42d177dfb9d6ce350389583
cfa71484eb8ae614c/Implementations/content/graphs%20(12)/
Advanced/DirectedMST.h
 * for the reconstruction Description: Finds a minimum spanning
 * tree/arborescence of a directed graph, given a root node. If
no MST exists,
 * returns -1. Time:  $O(E \log V)$  Status: Stress-tested, also
tested on NWERC
 * 2018 fastestspeedrun
 */
#pragma once

#include "../data-structures/UnionFindRollback.h"

struct Edge {
    int a, b;
    ll w;
};

struct Node { /// lazy skew heap node
    Edge key;
    Node *l, *r;
    ll delta;
    void prop() {

```

```

        key.w += delta;
        if (l)
            l->delta += delta;
        if (r)
            r->delta += delta;
        delta = 0;
    }
    Edge top() {
        prop();
        return key;
    }
};

Node *merge(Node *a, Node *b) {
    if (!a || !b)
        return a ? b;
    a->prop(), b->prop();
    if (a->key.w > b->key.w)
        swap(a, b);
    swap(a->l, (a->r = merge(b, a->r)));
    return a;
}

void pop(Node *&a) {
    a->prop();
    a = merge(a->l, a->r);
}

pair<ll, vi> dmst(int n, int r, vector<Edge> &g) {
    RollbackUF uf(n);
    vector<Node *> heap(n);
    for (Edge e : g)
        heap[e.b] = merge(heap[e.b], new Node{e});
    ll res = 0;
    vi seen(n, -1), path(n), par(n);
    seen[r] = r;
    vector<Edge> Q(n), in(n, {-1, -1}), comp;
    deque<tuple<int, int, vector<Edge>>> cys;

```

```

rep(s, 0, n) {
    int u = s, qi = 0, w;
    while (seen[u] < 0) {
        if (!heap[u])
            return {-1, {}};
        Edge e = heap[u]->top();
        heap[u]->delta -= e.w, pop(heap[u]);
        Q[qi] = e, path[qi++] = u, seen[u] = s;
        res += e.w, u = uf.find(e.a);
        if (seen[u] == s) { /// found cycle, contract
            Node *cyc = 0;
            int end = qi, time = uf.time();
            do
                cyc = merge(cyc, heap[w = path[--qi]]);
            while (uf.join(u, w));
            u = uf.find(u), heap[u] = cyc, seen[u] = -1;
            cycs.push_front({u, time, {&Q[qi], &Q[end]}});
        }
    }
    rep(i, 0, qi) in[uf.find(Q[i].b)] = Q[i];
}

for (auto &[u, t, comp] : cycs) { // restore sol (optional)
    uf.rollback(t);
    Edge inEdge = in[u];
    for (auto &e : comp)
        in[uf.find(e.b)] = e;
    in[uf.find(inEdge.b)] = inEdge;
}
rep(i, 0, n) par[i] = in[i].a;
return {res, par};
}

```

3.4. EdgeColoring.h

```

/**
 * Author: Simon Lindholm

```

```

 * Date: 2020-10-12
 * License: CC0
 * Source: https://en.wikipedia.org/wiki/Misra\_%26\_Gries\_edge\_coloring\_algorithm
 * https://codeforces.com/blog/entry/75431 for the note about bipartite graphs.
 * Description: Given a simple, undirected graph with max degree $D$, computes a
 * $(D + 1)$-coloring of the edges such that no neighboring edges share a color.
 * ($D$-coloring is NP-hard, but can be done for bipartite graphs by repeated
 * matchings of max-degree nodes.) Time: $O(NM)$ Status: stress-tested, tested on
 * kattis:gamescheduling
 */
#pragma once

```

```

vi edgeColoring(int N, vector<pii> eds) {
    vi cc(N + 1), ret(sz(eds)), fan(N), free(N), loc;
    for (pii e : eds)
        ++cc[e.first], ++cc[e.second];
    int u, v, ncols = *max_element(all(cc)) + 1;
    vector<vi> adj(N, vi(ncols, -1));
    for (pii e : eds) {
        tie(u, v) = e;
        fan[0] = v;
        loc.assign(ncols, 0);
        int at = u, end = u, d, c = free[u], ind = 0, i = 0;
        while (d = free[v], !loc[d] && (v = adj[u][d]) != -1)
            loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
        cc[loc[d]] = c;
        for (int cd = d; at != -1; cd ^= c ^ d, at = adj[at][cd])
            swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);
        while (adj[fan[i]][d] != -1) {
            int left = fan[i], right = fan[++i], e = cc[i];

```

```

    adj[u][e] = left;
    adj[left][e] = u;
    adj[right][e] = -1;
    free[right] = e;
}
adj[u][d] = fan[i];
adj[fan[i]][d] = u;
for (int y : {fan[0], u, end})
    for (int &z = free[y] = 0; adj[y][z] != -1; z++)
        ;
}
rep(i, 0, sz(eds)) for (tie(u, v) = eds[i]; adj[u][ret[i]] !=
v; ++ret[i];
return ret;
}

```

3.5. EulerWalk.h

```

/**
 * Author: Simon Lindholm
 * Date: 2019-12-31
 * License: CC0
 * Source: folklore
 * Description: Eulerian undirected/directed path/cycle
algorithm.
 * Input should be a vector of (dest, global edge index), where
 * for undirected graphs, forward/backward edges have the same
index.
 * Returns a list of nodes in the Eulerian path/cycle with src
at both start and
 * end, or empty list if no cycle/path exists. To get edge
indices back, add
 * .second to s and ret. Time: O(V + E) Status: stress-tested
 */
#pragma once

vi eulerWalk(vector<vector<pii>> &gr, int nedges, int src = 0)

```

```

{
    int n = sz(gr);
    vi D(n), its(n), eu(nedges), ret, s = {src};
    D[src]++; // to allow Euler paths, not just cycles
    while (!s.empty()) {
        int x = s.back(), y, e, &it = its[x], end = sz(gr[x]);
        if (it == end) {
            ret.push_back(x);
            s.pop_back();
            continue;
        }
        tie(y, e) = gr[x][it++];
        if (!eu[e]) {
            D[x]--, D[y]++;
            eu[e] = 1;
            s.push_back(y);
        }
    }
    for (int x : D)
        if (x < 0 || sz(ret) != nedges + 1)
            return {};
    return {ret.rbegin(), ret.rend()};
}

```

3.6. HLD.h

```

struct HLD {
    int n, timer = 0;
    vi top, tin, p, sub;
    HLD(vvi &adj) : n(sz(adj)), top(n), tin(n), p(n, -1), sub(n,
1) {
        vi ord(n + 1);
        for (int i = 0, t = 0, v = ord[i]; i < n; v = ord[++i])
            for (auto &to : adj[v])
                if (to != p[v])
                    p[to] = v, ord[++t] = to;
        for (int i = n - 1, v = ord[i]; i > 0; v = ord[--i])

```

```

    sub[p[v]] += sub[v];
    for (int v = 0; v < n; v++)
        if (sz(adj[v]))
            iter_swap(begin(adj[v]), max_element(all(adj[v]), [&
(int a, int b) {
                return make_pair(a != p[v], sub[a]) <
                    make_pair(b != p[v], sub[b]);
            }));
function<void(int)> dfs = [&](int v) {
    tin[v] = timer++;
    for (auto &to : adj[v])
        if (to != p[v]) {
            top[to] = (to == adj[v][0] ? top[v] : to);
            dfs(to);
        }
};
dfs(0);
}
int lca(int u, int v) {
    return process(u, v, [](int, int) {});
}
template <class B> int process(int a, int b, B op, bool
ignore_lca = false) {
    for (int v;; op(tin[v], tin[b]), b = p[v]) {
        if (tin[a] > tin[b])
            swap(a, b);
        if ((v = top[b]) == top[a])
            break;
    }
    if (int l = tin[a] + ignore_lca, r = tin[b]; l <= r)
        op(l, r);
    return a;
}
template <class B> void subtree(int v, B op, bool ignore_lca
= false) {
    if (sub[v] > 1 or !ignore_lca)

```

```

        op(tin[v] + ignore_lca, tin[v] + sub[v] - 1);
    }
};

3.7. KthAnc.h
// O(log n) LCA with Kth anc
struct LCA {
    int n;
    vvi &adjLists;
    int lg;
    vvi up;
    vi depth;
    LCA(vvi &adjLists, int root = 0) : n(sz(_adjLists)),
adjLists(_adjLists) {
        lg = 1;
        int pw = 1;
        while (pw <= n)
            pw <= 1, lg++;
        // lg = 20
        up = vvi(n, vi(lg));
        depth.assign(n, -1);
        function<void(int, int)> parentDFS = [&](int from, int
parent) {
            depth[from] = depth[parent] + 1;
            up[from][0] = parent;
            for (auto to : adjLists[from]) {
                if (to == parent)
                    continue;
                parentDFS(to, from);
            }
        };
        parentDFS(root, root);
        for (int j = 1; j < lg; j++) {
            for (int i = 0; i < n; i++) {
                up[i][j] = up[up[i][j - 1]][j - 1];
            }
        }
    }
};

```

```

    }
}
int kthAnc(int v, int k) {
    int ret = v;
    int pw = 0;
    while (k) {
        if (k & 1)
            ret = up[ret][pw];
        k >>= 1;
        pw++;
    }
    return ret;
}
int lca(int u, int v) {
    if (depth[u] > depth[v])
        swap(u, v);
    v = kthAnc(v, depth[v] - depth[u]);
    if (u == v)
        return v;
    while (up[u][0] != up[v][0]) {
        int i = 0;
        for (; i < lg - 1; i++) {
            if (up[u][i + 1] == up[v][i + 1])
                break;
        }
        u = up[u][i], v = up[v][i];
    }
    return up[u][0];
};
int dist(int u, int v) { return depth[u] + depth[v] - 2 *
depth[lca(u, v)]; }
};

```

3.8. LCA.h

```

// O(1) LCA
struct LCA {

```

```

    int T = 0;
    vi st, path, ret;
    vi en, d;
    RMQ<int> rmq;
    LCA(vector<vi> &C)
        : st(sz(C)), en(sz(C)), d(sz(C)), rmq((dfs(C, 0, -1),
ret)) {}
    void dfs(vvi &adj, int v, int par) {
        st[v] = T++;
        for (auto to : adj[v])
            if (to != par) {
                path.pb(v), ret.pb(st[v]);
                d[to] = d[v] + 1;
                dfs(adj, to, v);
            }
        en[v] = T - 1;
    }
    bool anc(int p, int c) { return st[p] <= st[c] and en[p] >=
en[c]; }
    int lca(int a, int b) {
        if (a == b)
            return a;
        tie(a, b) = minmax(st[a], st[b]);
        return path[rmq.query(a, b - 1)];
    }
    int dist(int a, int b) { return d[a] + d[b] - 2 * d[lca(a,
b)]; }
};

```

3.9. MaximumClique.h

```

/**
 * Author: chilli, SJTU, Janez Konc
 * Date: 2019-05-10
 * License: GPL3+
 * Source: https://en.wikipedia.org/wiki/MaxCliqueDyn\_maximum\_
clique\_algorithm,

```

```

* https://gitlab.com/janezkonc/mcqd/blob/master/mcqd.h
Description: Quickly
* finds a maximum clique of a graph (given as symmetric bitset
matrix;
* self-edges not allowed). Can be used to find a maximum
independent set by
* finding a clique of the complement graph. Time: Runs in
about 1s for n=155
* and worst case random graphs (p=.90). Runs faster for sparse
graphs. Status:
* stress-tested
*/
typedef vector<bitset<200>> vb;
struct Maxclique {
    double limit = 0.025, pk = 0;
    struct Vertex {
        int i, d = 0;
    };
    typedef vector<Vertex> vv;
    vb e;
    vv V;
    vector<vi> C;
    vi qmax, q, S, old;
    void init(vv &r) {
        for (auto &v : r)
            v.d = 0;
        for (auto &v : r)
            for (auto j : r)
                v.d += e[v.i][j.i];
        sort(all(r), [](auto a, auto b) { return a.d > b.d; });
        int mxD = r[0].d;
        rep(i, 0, sz(r)) r[i].d = min(i, mxD) + 1;
    }
    void expand(vv &R, int lev = 1) {
        S[lev] += S[lev - 1] - old[lev];
        old[lev] = S[lev - 1];
    }

```

```

while (sz(R)) {
    if (sz(q) + R.back().d <= sz(qmax))
        return;
    q.push_back(R.back().i);
    vv T;
    for (auto v : R)
        if (e[R.back().i][v.i])
            T.push_back({v.i});
    if (sz(T)) {
        if (S[lev]++ / ++pk < limit)
            init(T);
        int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1, 1);
        C[1].clear(), C[2].clear();
        for (auto v : T) {
            int k = 1;
            auto f = [&](int i) { return e[v.i][i]; };
            while (any_of(all(C[k]), f))
                k++;
            if (k > mxk)
                mxk = k, C[mxk + 1].clear();
            if (k < mnk)
                T[j++].i = v.i;
            C[k].push_back(v.i);
        }
        if (j > 0)
            T[j - 1].d = 0;
        rep(k, mnk, mxk + 1) for (int i : C[k]) T[j].i = i,
T[j++].d = k;
        expand(T, lev + 1);
    } else if (sz(q) > sz(qmax))
        qmax = q;
    q.pop_back(), R.pop_back();
}
}
vi maxClique() {
    init(V), expand(V);

```

```

    return qmax;
}
Maxclique(vb conn) : e(conn), C(sz(e) + 1), S(sz(C)), old(S)
{
    rep(i, 0, sz(e)) V.push_back({i});
}
};

```

3.10. MaximumIndependentSet.h

```

/**
 * Author: chilli
 * Date: 2019-05-17
 * Source: Wikipedia
 * Description: To obtain a maximum independent set of a graph,
find a max
 * clique of the complement. If the graph is bipartite, see
MinimumVertexCover.
 */

```

3.11. MinCostMaxFlow.h

```

template <const int MAX_N, typename flow_t, typename cost_t,
flow_t FLOW_INF,
        cost_t COST_INF, const int SCALE = 16>
struct CostScalingMCMF {
#define sz(a) a.size()
#define zero_stl(v, sz) fill(v.begin(), v.begin() + (sz), 0)
    struct Edge {
        int v;
        flow_t c;
        cost_t d;
        int r;
        Edge() = default;
        Edge(int v, flow_t c, cost_t d, int r) : v(v), c(c), d(d),
r(r) {}
    };
    vector<Edge> g[MAX_N];

```

```

cost_t negativeSelfLoop;
array<cost_t, MAX_N> pi, excess;
array<int, MAX_N> level, ptr;
CostScalingMCMF() { negativeSelfLoop = 0; }
void clear() {
    negativeSelfLoop = 0;
    for (int i = 0; i < MAX_N; i++)
        g[i].clear();
}
void addEdge(int s, int e, flow_t cap, cost_t cost) {
    if (s == e) {
        if (cost < 0)
            negativeSelfLoop += cap * cost;
        return;
    }
    g[s].push_back(Edge(e, cap, cost, sz(g[e])));
    g[e].push_back(Edge(s, 0, -cost, sz(g[s]) - 1));
}
flow_t getMaxFlow(int V, int S, int T) {
    auto BFS = [&]() {
        zero_stl(level, V);
        queue<int> q;
        q.push(S);
        level[S] = 1;
        for (q.push(S); !q.empty(); q.pop()) {
            int v = q.front();
            for (const auto &e : g[v])
                if (!level[e.v] && e.c)
                    q.push(e.v), level[e.v] = level[v] + 1;
        }
        return level[T];
    };
    function<flow_t(int, flow_t)> DFS = [&](int v, flow_t fl) {
        if (v == T || fl == 0)
            return fl;
        for (int &i = ptr[v]; i < (int)g[v].size(); i++) {

```

```

    Edge &e = g[v][i];
    if (level[e.v] != level[v] + 1 || !e.c)
        continue;
    flow_t delta = DFS(e.v, min(fl, e.c));
    if (delta) {
        e.c -= delta;
        g[e.v][e.r].c += delta;
        return delta;
    }
}
return flow_t(0);
};
flow_t maxFlow = 0, tmp = 0;
while (BFS()) {
    zero_stl(ptr, V);
    while ((tmp = DFS(S, FLOW_INF)))
        maxFlow += tmp;
}
return maxFlow;
}
pair<flow_t, cost_t> maxflow(int N, int S, int T) {
    flow_t maxFlow = 0;
    cost_t eps = 0, minCost = 0;
    stack<int, vector<int>> stk;
    auto c_pi = [&](int v, const Edge &edge) {
        return edge.d + pi[v] - pi[edge.v];
    };
    auto push = [&](int v, Edge &edge, flow_t delta, bool flag)
{
    delta = min(delta, edge.c);
    edge.c -= delta;
    g[edge.v][edge.r].c += delta;
    excess[v] -= delta;
    excess[edge.v] += delta;
    if (flag && 0 < excess[edge.v] && excess[edge.v] <=
delta)

```

```

        stk.push(edge.v);
    };
    auto relabel = [&](int v, cost_t delta) { pi[v] -= delta +
eps; };
    auto lookAhead = [&](int v) {
        if (excess[v])
            return false;
        cost_t delta = COST_INF;
        for (auto &e : g[v]) {
            if (e.c <= 0)
                continue;
            cost_t cp = c_pi(v, e);
            if (cp < 0)
                return false;
            else
                delta = min(delta, cp);
        }
        relabel(v, delta);
        return true;
    };
    auto discharge = [&](int v) {
        cost_t delta = COST_INF;
        for (int i = 0; i < sz(g[v]); i++) {
            Edge &e = g[v][i];
            if (e.c <= 0)
                continue;
            cost_t cp = c_pi(v, e);
            if (cp < 0) {
                if (lookAhead(e.v)) {
                    i--;
                    continue;
                }
                push(v, e, excess[v], true);
                if (excess[v] == 0)
                    return;
            } else

```



```

        delta = min(delta, cp);
    }
    relabel(v, delta);
    stk.push(v);
};
zero_stl(pi, N);
zero_stl(excess, N);
for (int i = 0; i < N; i++)
    for (auto &e : g[i])
        minCost += e.c * e.d, e.d *= MAX_N + 1, eps = max(eps,
e.d);
maxFlow = getMaxFlow(N, S, T);
while (eps > 1) {
    eps /= SCALE;
    if (eps < 1)
        eps = 1;
    stk = stack<int, vector<int>>>();
    for (int v = 0; v < N; v++)
        for (auto &e : g[v])
            if (c_pi(v, e) < 0 && e.c > 0)
                push(v, e, e.c, false);
    for (int v = 0; v < N; v++)
        if (excess[v] > 0)
            stk.push(v);
    while (stk.size()) {
        int top = stk.top();
        stk.pop();
        discharge(top);
    }
}
for (int v = 0; v < N; v++)
    for (auto &e : g[v])
        e.d /= MAX_N + 1, minCost -= e.c * e.d;
minCost = minCost / 2 + negativeSelfLoop;
return {maxFlow, minCost};
}

```

```

};

void solve() {
    CostScalingMCMF<102, int, int, 100, 100> flow;
    int n, m;
    cin >> n >> m;
    int start = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            int inp;
            cin >> inp;
            if (inp) {
                flow.addEdge(i + 1, n + 1 + j, 1, 0);
                start++;
            } else
                flow.addEdge(i + 1, n + 1 + j, 1, 1);
        }
    }
    int counta = 0, countb = 0;
    for (int i = 0; i < n; i++) {
        int inp;
        cin >> inp;
        counta += inp;
        flow.addEdge(0, i + 1, inp, 0);
    }
    for (int i = 0; i < m; i++) {
        int inp;
        cin >> inp;
        countb += inp;
        flow.addEdge(n + i + 1, n + m + 1, inp, 0);
    }
    if (counta != countb) {
        cout << -1 << endl;
        return;
    }
    pii t = flow.maxflow(102, 0, n + m + 1);
}

```

```

    if (t.first != counta) {
        cout << -1 << endl;
        return;
    }
    cout << t.second + start + t.second - counta << endl;
}

```

3.12. MinimumVertexCover.h

```

/**
 * Author: Johan Sannemo, Simon Lindholm
 * Date: 2016-12-15
 * License: CC0
 * Description: Finds a minimum vertex cover in a bipartite
graph.
 * The size is the same as the size of a maximum matching, and
 * the complement is a maximum independent set.
 * Status: stress-tested
 */
#pragma once

#include "DFSMatching.h"

vi cover(vector<vi> &g, int n, int m) {
    vi match(m, -1);
    int res = dfsMatching(g, match);
    vector<bool> lfound(n, true), seen(m);
    for (int it : match)
        if (it != -1)
            lfound[it] = false;
    vi q, cover;
    rep(i, 0, n) if (lfound[i]) q.push_back(i);
    while (!q.empty()) {
        int i = q.back();
        q.pop_back();
        lfound[i] = 1;
        for (int e : g[i])

```

```

        if (!seen[e] && match[e] != -1) {
            seen[e] = true;
            q.push_back(match[e]);
        }
    }
    rep(i, 0, n) if (!lfound[i]) cover.push_back(i);
    rep(i, 0, m) if (seen[i]) cover.push_back(n + i);
    assert(sz(cover) == res);
    return cover;
}

```

3.13. SCC.h

```

struct SCC {
    int n;
    vi val, cc, z;
    vvi comps;
    SCC(vvi &adj) : n(sz(adj)), val(n), cc(n, -1) {
        int timer = 0;
        function<int(int)> dfs = [&](int x) {
            int low = val[x] = ++timer, b;
            z.push_back(x);
            for (auto y : adj[x])
                if (cc[y] < 0)
                    low = min(low, val[y] ? dfs(y));

            if (low == val[x]) {
                comps.push_back(vi());
                do {
                    b = z.back();
                    z.pop_back();
                    comps.back().push_back(b);
                    cc[b] = sz(comps) - 1;
                } while (x != b);
            }
            return val[x] = low;
        };
    }
};

```

```

    for (int i = 0; i < n; i++)
        if (cc[i] < 0)
            dfs(i);
}
int operator()(int i) { return cc[i]; }
int size(int i) { return sz(comps[cc[i]]); }
};

```

3.14. WeightedMatching.h

```

/**
 * Author: Benjamin Qi, chilli
 * Date: 2020-04-04
 * License: CC0
 * Source:
 * https://github.com/bqi343/USACO/blob/master/Implementations/
content/graphs%20(12)/Matching/Hungarian.h
 * Description: Given a weighted bipartite graph, matches every
node on
 * the left with a node on the right such that no
 * nodes are in two matchings and the sum of the edge weights
is minimal. Takes
 * cost[N][M], where cost[i][j] = cost for L[i] to be matched
with R[j] and
 * returns (min cost, match), where L[i] is matched with
 * R[match[i]]. Negate costs for max cost. Requires  $N \leq M$ .
 * Time:  $O(N^2M)$ 
 * Status: Tested on kattis:cordonbleu, stress-tested
 */
#pragma once

pair<int, vi> hungarian(const vector<vi> &a) {
    if (a.empty())
        return {0, {}};
    int n = sz(a) + 1, m = sz(a[0]) + 1;
    vi u(n), v(m), p(m), ans(n - 1);
    rep(i, 1, n) {

```

```

        p[0] = i;
        int j0 = 0; // add "dummy" worker 0
        vi dist(m, INT_MAX), pre(m, -1);
        vector<bool> done(m + 1);
        do { // dijkstra
            done[j0] = true;
            int i0 = p[j0], j1, delta = INT_MAX;
            rep(j, 1, m) if (!done[j]) {
                auto cur = a[i0 - 1][j - 1] - u[i0] - v[j];
                if (cur < dist[j])
                    dist[j] = cur, pre[j] = j0;
                if (dist[j] < delta)
                    delta = dist[j], j1 = j;
            }
            rep(j, 0, m) {
                if (done[j])
                    u[p[j]] += delta, v[j] -= delta;
                else
                    dist[j] -= delta;
            }
            j0 = j1;
        } while (p[j0]);
        while (j0) { // update alternating path
            int j1 = pre[j0];
            p[j0] = p[j1], j0 = j1;
        }
        rep(j, 1, m) if (p[j]) ans[p[j] - 1] = j - 1;
        return {-v[0], ans}; // min cost
    }
}

```

3.15. bellman.h

```

bool bellman_ford(int n, int src, vector<vector<pair<int,
int>>> &adj,
                    vector<long long> &dist) {
    const long long INF = 1e18;

```

```

dist.assign(n, INF);
dist[src] = 0;
for (int i = 0; i < n - 1; i++) {
    for (int u = 0; u < n; u++) {
        if (dist[u] == INF)
            continue;
        for (auto &p : adj[u]) {
            int v = p.first, w = p.second;
            if (dist[u] + w < dist[v])
                dist[v] = dist[u] + w;
        }
    }
}
for (int u = 0; u < n; u++) {
    if (dist[u] == INF)
        continue;
    for (auto &p : adj[u]) {
        int v = p.first, w = p.second;
        if (dist[u] + w < dist[v])
            return false;
    }
}
return true;
}

```

3.16. bridges.h

```

int n; // number of nodes
vector<vector<int>> adj; // adjacency list of graph

vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;

```

```

    for (int to : adj[v]) {
        if (to == p)
            continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] > tin[v])
                IS_BRIDGE(v, to);
        }
    }
}

void find_bridges() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs(i);
    }
}

```

// ARTICULATION POINTS:

```

int n;
vector<vector<int>> adj;
vector<bool> visited;
vector<int> tin, low;
int timer;
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children = 0;
    for (int to : adj[v]) {

```

```

    if (to == p)
        continue;
    if (visited[to]) {
        low[v] = min(low[v], tin[to]);
    } else {
        dfs(to, v);
        low[v] = min(low[v], low[to]);
        if (low[to] >= tin[v] && p != -1)
            IS_CUTPOINT(v);
        ++children;
    }
}
if (p == -1 && children > 1)
    IS_CUTPOINT(v);
}

void find_cutpoints() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs(i);
    }
}

// arya bridges
void findBridges_dfs(int u, int p, int &time,
    vector<vector<int>> &adj,
    vector<int> &disc, vector<int> &low,
    vector<pair<int, int>> &bridges) {
    disc[u] = low[u] = time++;

    for (int v : adj[u]) {
        if (v == p)
            continue;

```

```

        if (disc[v] != -1) {
            low[u] = min(low[u], disc[v]);
        } else {
            findBridges_dfs(v, u, time, adj, disc, low, bridges);
            low[u] = min(low[u], low[v]);
            if (low[v] > disc[u]) {
                bridges.push_back({u, v});
            }
        }
    }
}

vector<pair<int, int>> findBridges(int n, vector<vector<int>>
&adj) {
    vector<int> disc(n, -1), low(n, -1);
    vector<pair<int, int>> bridges;
    int time = 0;

    for (int i = 0; i < n; ++i) {
        if (disc[i] == -1) {
            findBridges_dfs(i, -1, time, adj, disc, low, bridges);
        }
    }
    return bridges;
}

```

3.17. dijkstra.h

```

const int INF = 1000000000;
vector<vector<pair<int, int>>> adj;

void dijkstra(int s, vector<int> &d, vector<int> &p) {
    int n = adj.size();
    d.assign(n, INF);
    p.assign(n, -1);
    vector<bool> u(n, false);

```

```

d[s] = 0;
for (int i = 0; i < n; i++) {
    int v = -1;
    for (int j = 0; j < n; j++) {
        if (!u[j] && (v == -1 || d[j] < d[v]))
            v = j;
    }

    if (d[v] == INF)
        break;

    u[v] = true;
    for (auto edge : adj[v]) {
        int to = edge.first;
        int len = edge.second;

        if (d[v] + len < d[to]) {
            d[to] = d[v] + len;
            p[to] = v;
        }
    }
}
}

```

3.18. floyd_washall.h

```

const long long INF = (long long)1e18;
bool floyd_warshall(int n, vector<vector<long long>> &dist) {
    // initialise dist with edge weights, INF if no edge exists
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (dist[i][k] < INF && dist[k][j] < INF)
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k]
[j]);
}

```

```

    for (int i = 0; i < n; i++)
        if (dist[i][i] < 0)
            return true;
    return false;
}

```

3.19. hopcroftKarp.h

```

/**
 * Author: Chen Xing
 * Date: 2009-10-13
 * License: CC0
 * Source: N/A
 * Description: Fast bipartite matching algorithm. Graph $g$
should be a list
 * of neighbors of the left partition, and $btoa$ should be a
vector full of
 * -1's of the same size as the right partition. Returns the
size of
 * the matching. $btoa[i]$ will be the match for vertex $i$ on
the right side,
 * or $-1$ if it's not matched.
 * Usage: vi btoa(m, -1); hopcroftKarp(g, btoa);
 * Time: O(\sqrt{V}E)
 * Status: stress-tested by MinimumVertexCover, and tested on
 * oldkattis.adkbipmatch and SPOJ:MATCHING
 */

```

#pragma once

```

bool dfs(int a, int L, vector<vi> &g, vi &btoa, vi &A, vi &B) {
    if (A[a] != L)
        return 0;
    A[a] = -1;
    for (int b : g[a])
        if (B[b] == L + 1) {
            B[b] = 0;
            if (btoa[b] == -1 || dfs(btoa[b], L + 1, g, btoa, A, B))

```

```

        return btoa[b] = a, 1;
    }
    return 0;
}

int hopcroftKarp(vector<vi> &g, vi &btoa) {
    int res = 0;
    vi A(g.size()), B(btoa.size()), cur, next;
    for (;;) {
        fill(all(A), 0);
        fill(all(B), 0);
        /// Find the starting nodes for BFS (i.e. layer 0).
        cur.clear();
        for (int a : btoa)
            if (a != -1)
                A[a] = -1;
        rep(a, 0, sz(g)) if (A[a] == 0) cur.push_back(a);
        /// Find all layers using bfs.
        for (int lay = 1;; lay++) {
            bool islast = 0;
            next.clear();
            for (int a : cur)
                for (int b : g[a]) {
                    if (btoa[b] == -1) {
                        B[b] = lay;
                        islast = 1;
                    } else if (btoa[b] != a && !B[b]) {
                        B[b] = lay;
                        next.push_back(btoa[b]);
                    }
                }
            if (islast)
                break;
            if (next.empty())
                return res;
            for (int a : next)

```

```

        A[a] = lay;
        cur.swap(next);
    }
    /// Use DFS to scan for augmenting paths.
    rep(a, 0, sz(g)) res += dfs(a, 0, g, btoa, A, B);
}
}

```

4. Number Theory

4.1. CRT.h

```

/**
 * Author: Simon Lindholm
 * Date: 2019-05-22
 * License: CC0
 * Description: Chinese Remainder Theorem.
 *
 * \texttt{crt(a, m, b, n)} computes  $x$  such that  $x \equiv a \pmod m$ ,  $x \equiv b \pmod n$ . If  $|a| < m$  and  $|b| < n$ ,  $x$  will obey  $0 \leq x < \text{lcm}(m, n)$ . Assumes  $mn < 2^{62}$ . Time:  $\log(n)$ 
 Status: Works
 */
#pragma once

#include "euclid.h"

ll crt(ll a, ll m, ll b, ll n) {
    if (n > m)
        swap(a, b), swap(m, n);
    ll x, y, g = euclid(m, n, x, y);
    assert((a - b) % g == 0); // else no solution
    x = (b - a) % n * x % n / g * m + a;
    return x < 0 ? x + m * n / g : x;
}

```

4.2. Factor.h

```

/**
 * Author: chilli, SJTU, pajenegod
 * Date: 2020-03-04
 * License: CC0
 * Source: own
 * Description: Pollard-rho randomized factorization algorithm.
Returns prime
 * factors of a number, in arbitrary order (e.g. 2299 -> \{11,
19, 11\}).
 * Time:  $O(n^{\{1/4\}})$ , less for numbers with small factors.
 * Status: stress-tested
 *
 * Details: This implementation uses the improvement described
here
 * (https://en.wikipedia.org/wiki/Pollard%27s\_rho\_algorithm#Variants), where
 * one can accumulate gcd calls by some factor (40 chosen here
through
 * exhaustive testing). This improves performance by
approximately 6-10x
 * depending on the inputs and speed of gcd. Benchmark found
here:
 * (https://ideone.com/nGGD9T)
 *
 * GCD can be improved by a factor of 1.75x using Binary GCD
 * (https://lemire.me/blog/2013/12/26/fastest-way-to-compute-the-greatest-common-divisor/).
 * However, with the gcd accumulation the bottleneck moves from
the gcd calls
 * to the modmul. As GCD only constitutes ~12% of runtime,
speeding it up
 * doesn't matter so much.
 *
 * This code can probably be sped up by using a faster mod mul
- potentially
 * montgomery reduction on 128 bit integers.
 * Alternatively, one can use a quadratic sieve for an
asymptotic improvement,
 * which starts being faster in practice around  $1e13$ .
 *
 * Brent's cycle finding algorithm was tested, but doesn't
reduce modmul calls
 * significantly.
 *
 * Subtle implementation notes:
 * - prd starts off as 2 to handle the case  $n = 4$ ; it's
harmless for other  $n$ 
 * since we're guaranteed that  $n > 2$ . (Pollard rho has
problems with prime
 * powers in general, but all larger ones happen to work.)
 * - t starts off as 30 to make the first gcd check come
earlier, as an
 * optimization for small numbers.
 * - we vary f between restarts because the cycle finding
algorithm does not
 * find the first element in the cycle but rather one at
distance  $k * |cycle|$ 
 * from the start, and that can result in continual failures
if all cycles
 * have the same size for all prime factors. E.g. fixing  $f(x) = x^2 + 1$  would
 * loop infinitely for  $n = 352523 * 352817$ , where all cycles
have size 821.
 * - we operate on residues in  $[i, n + i)$  which modmul is not
designed to
 * handle, but specifically  $modmul(x, x)$  still turns out to
work for small
 * enough  $i$ . (With reference to the proof in modmul-
proof.tex, the argument
 * for " $S$  is in  $[-c, 2c)$ " goes through unchanged, while  $S < 2^{63}$  now follows

```



```

*   from  $S < 2c$  and  $S = x^2 \pmod{c}$  together implying  $S < c + i^2$ .)
*/
#pragma once

#include "MillerRabin.h"
#include "ModMulLL.h"

ull pollard(ull n) {
    ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    auto f = [&](ull x) { return modmul(x, x, n) + i; };
    while (t++ % 40 || __gcd(prd, n) == 1) {
        if (x == y)
            x = ++i, y = f(x);
        if ((q = modmul(prd, max(x, y) - min(x, y), n)))
            prd = q;
        x = f(x), y = f(f(y));
    }
    return __gcd(prd, n);
}

vector<ull> factor(ull n) {
    if (n == 1)
        return {};
    if (isPrime(n))
        return {n};
    ull x = pollard(n);
    auto l = factor(x), r = factor(n / x);
    l.insert(l.end(), all(r));
    return l;
}

```

4.3. MillerRabin.h

```

u64 mult(u64 a, u64 b, u64 m = M) {
    i64 ret = a * b - m * (u64)(1.L / m * a * b);
    return ret + m * (ret < 0) - m * (ret >= (i64)m);
}

```

```

u64 pw(u64 b, u64 e, u64 m = M) {
    u64 ret = 1;
    for (; e; b = mult(b, b, m), e >>= 1)
        if (e & 1)
            ret = mult(ret, b, m);
    return ret;
}

bool isPrime(u64 n) { // deterministic upto  $7e^{18}$ 
    if (n < 2 || n % 6 % 4 != 1)
        return (n | 1) == 3;
    u64 A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
        s = __builtin_ctzll(n - 1), d = n >> s;
    for (u64 a : A) {
        u64 p = pw(a % n, d, n), i = s;
        while (p != 1 && p != n - 1 && a % n && i--)
            p = mult(p, p, n);
        if (p != n - 1 && i != s)
            return 0;
    }
    return 1;
}

```

4.4. ModularArithmetic.h

```

int add(int x, int y, int m = M) {
    int ret = (x + y) % m;
    if (ret < 0)
        ret += m;
    return ret;
}

int mult(int x, int y, int m = M) {
    int ret = (x * y) % m;
    if (ret < 0)
        ret += m;
    return ret;
}

int pw(int a, int b, int m = M) {

```

```

int ret = 1;
int p = a;
while (b) {
    if (b & 1)
        ret = mult(ret, p, m);
    b >>= 1;
    p = mult(p, p, m);
}
return ret;
}

#define LL int
const long long mod = 1e9 + 7;

int euclid(int a, int b, int &x, int &y) {
    if (!b)
        return x = 1, y = 0, a;
    int d = euclid(b, a % b, y, x);
    return y -= a / b * x, d;
}

int modulo_inverse(int a, int m) {
    int x, y;
    int g = euclid(a, m, x, y);
    if (g != 1) {
        return -1;
    } else {
        x = (x % m + m) % m;
        return x;
    }
}

LL mod_mul(LL a, LL b) {
    a = a % mod;
    b = b % mod;
    return ((a * b) % mod) + mod) % mod;

```

```

}

LL mod_add(LL a, LL b) {
    a = a % mod;
    b = b % mod;
    return ((a + b) % mod) + mod) % mod;
}

const int MX = 5e5 + 1;
vector<int> inv(MX + 1), fci(MX + 1), fc(MX + 1);
const int Mod = 1e9 + 7;

void Inverses() {
    inv[1] = 1;
    for (int i = 2; i <= MX; i++) {
        inv[i] = Mod - Mod / i * inv[Mod % i] % Mod;
    }
}

void Factorials() {
    fc[0] = fc[1] = 1;
    for (int i = 2; i <= MX; i++) {
        fc[i] = fc[i - 1] * i % Mod;
    }
}

void InverseFactorials() {
    Inverses();
    Factorials();
    fci[1] = fci[0] = 1;
    for (int i = 2; i <= MX; i++) {
        fci[i] = fci[i - 1] * inv[i] % Mod;
    }
}

int nck(int num, int k) {

```

```

    if (num < 0) {
        return 0;
    }
    if (k < 0) {
        return 0;
    }
    if (num < k) {
        return 0;
    } else {
        return fc[num] * fci[k] % Mod * fci[num - k] % Mod;
    }
}

int BinExpItermod(int a, int b) {
    int ans = 1;
    while (b > 0) {
        if (b & 1) {
            ans = (ans * a) % mod;
        }
        a = (a * a) % mod;
        b = b >> 1;
    }
    return ans;
}

```

4.5. bit_bns.h

```
// --- Bit Binary Search in o(log(n)) ---
```

```
const int M = 20 const int N = 1 << M
```

```

                                int
                                lower_bound(int val) {

int ans = 0, sum = 0;
for (int i = M - 1; i >= 0; i--) {
    int x = ans + (1 << i);
    if (sum + bit[x] < val)
        ans = x, sum += bit[x];
}

```

```

    }

    return ans + 1;
}

```

4.6. matrix_expo.h

```

int **matrixmul(int **matrix1, int **matrix2) {
    int **matrix3 = new int *[2];
    for (int i = 0; i < 2; i++)
        matrix3[i] = new int[2];

    matrix3[0][0] =
        (matrix1[0][0] * matrix2[0][0]) + (matrix1[0][1] *
matrix2[1][0]);
    matrix3[0][1] =
        (matrix1[0][0] * matrix2[0][1]) + (matrix1[0][1] *
matrix2[1][1]);
    matrix3[1][0] =
        (matrix1[1][0] * matrix2[0][0]) + (matrix1[1][1] *
matrix2[1][0]);
    matrix3[1][1] =
        (matrix1[1][0] * matrix2[0][1]) + (matrix1[1][1] *
matrix2[1][1]);
    matrix3[0][0] %= M;
    matrix3[1][0] %= M;
    matrix3[0][1] %= M;
    matrix3[1][1] %= M;

    return matrix3;
}

```

```

int **matrixexpo(int **matrix, int n, int **ans) {
    while (n > 0) {
        if (n % 2 == 1)
            ans = matrixmul(ans, matrix);
        matrix = matrixmul(matrix, matrix);
    }
}

```

```

    n /= 2;
}
return ans;
}

```

4.7. phiFunction.h

```

/**
 * Author: Håkan Terelius
 * Date: 2009-09-25
 * License: CC0
 * Source: http://en.wikipedia.org/wiki/Euler's_totient_
function
 * Description: \emph{Euler's  $\phi$ } function is defined as  $\phi(n)$ 
\phi(n):=\#\$ of
 * positive integers  $\leq n$  that are coprime with  $n$ .
 *  $\phi(1)=1$ ,  $p$  prime  $\Rightarrow \phi(p^k)=(p-1)p^{k-1}$ ,
 $m, n$  coprime
 *  $\Rightarrow \phi(mn)=\phi(m)\phi(n)$ . If  $n=p_1^{k_1}p_2^{k_2}\dots$ 
 $p_r^{k_r}$  then  $\phi(n) = (p_1-1)p_1^{k_1-1}\dots$ 
 $(p_r-1)p_r^{k_r-1}$ .
 *  $\phi(n)=n \cdot \prod_{p|n} (1-1/p)$ .
 *
 *  $\sum_{d|n} \phi(d) = n$ ,  $\sum_{1 \leq k \leq n, \gcd(k,n)=1} k = n$ 
 *  $\phi(n)/2$ ,  $n > 1$ 
 *
 * \textbf{Euler's thm}:  $a, n$  coprime  $\Rightarrow a^{\phi(n)} \equiv 1 \pmod{n}$ .
 *
 * \textbf{Fermat's little thm}:  $p$  prime  $\Rightarrow a^{p-1} \equiv 1 \pmod{p}$ 
 $\forall a$ . Status: Tested
 */
#pragma once

```

```

const int LIM = 5000000;
int phi[LIM];

void calculatePhi() {
    rep(i, 0, LIM) phi[i] = i & 1 ? i : i / 2;
    for (int i = 3; i < LIM; i += 2)
        if (phi[i] == i)
            for (int j = i; j < LIM; j += i)
                phi[j] -= phi[j] / i;
}

```

4.8. some_dp.h

```

// LIS
int lis(vector<int> const &a) {
    int n = a.size();
    const int INF = 1e9;
    vector<int> d(n + 1, INF);
    d[0] = -INF;

    for (int i = 0; i < n; i++) {
        int l = upper_bound(d.begin(), d.end(), a[i]) - d.begin();
        if (d[l - 1] < a[i] && a[i] < d[l])
            d[l] = a[i];
    }

    int ans = 0;
    for (int l = 0; l <= n; l++) {
        if (d[l] < INF)
            ans = l;
    }
    return ans;
}
// or segtree lol

```

4.9. spf.h

```
int MX = 1e7 + 1;
vi spf(MX + 1, INT32_MAX);
vector<int> is_prime(MX + 1, 1);
void sieve(int n = MX) {
    is_prime[0] = is_prime[1] = 0;
    int cnt = 1;
    for (int i = 2; i <= n; i++) {
        if (is_prime[i]) {
            for (int j = i * i; j <= n; j += i) {
                is_prime[j] = 0;
                spf[j] = min(i, spf[j]);
            }
            is_prime[i] = cnt;
            cnt++;
        }
    }
    return;
}
```

5. Strings

5.1. Manacher.h

```
/* Description: p[0][i] = half length of longest even
palindrome behind pos i,
p[1][i] = longest odd with center at pos i(half rounded down).
*/
array<vi, 2> manacher(const string &s) {
    int n = sz(s);
    array<vi, 2> p = {vi(n + 1), vi(n)};
    for (int z = 0; z < 2; z++)
        for (int i = 0, l = 0, r = 0; i < n; i++) {
            int t = r - i + !z;
            if (i < r)
                p[z][i] = min(t, p[z][l + t]);
            int L = i - p[z][i], R = i + p[z][i] - !z;
```

```
        while (L >= 1 && R + 1 < n && s[L - 1] == s[R + 1])
            p[z][i]++, L--, R++;
        if (R > r)
            l = L, r = R;
    }
    return p;
}
```

5.2. SuffixArray.h

```
/*Builds suffix array for a string.
\texttt{sa[i]} is the starting index of the suffix which
is  $i$ 'th in the sorted suffix array.
The returned vector is of size  $n+1$ , and  $\texttt{sa[0]} = n$ .
The  $\texttt{lcp}$  array contains longest common prefixes for
neighbouring strings in the suffix array:
 $\texttt{lcp[i]} = \texttt{lcp(sa[i], sa[i-1])}$ ,  $\texttt{lcp[0]} = 0$ .
The input string must not contain any zero bytes.
Time:  $O(n \log n)$ */
#define rep(i, j, k) for (int i = j; i < k; i++)
struct SuffixArray {
    vi sa, lcp;
    SuffixArray(string &s, int lim = 256) { // or
        basic_string<int>
            int n = sz(s) + 1, k = 0, a, b;
            vi x(all(s)), y(n), ws(max(n, lim));
            x.push_back(0), sa = lcp = y, iota(all(sa), 0);
            for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) {
                p = j, iota(all(y), n - j);
                rep(i, 0, n) if (sa[i] >= j) y[p++] = sa[i] - j;
                fill(all(ws), 0);
                rep(i, 0, n) ws[x[i]]++;
                rep(i, 1, lim) ws[i] += ws[i - 1];
                for (int i = n; i--;)
                    sa[--ws[x[y[i]]]] = y[i];
                swap(x, y), p = 1, x[sa[0]] = 0;
                rep(i, 1, n) a = sa[i - 1], b = sa[i],
```

```

        x[b] = (y[a] == y[b] && y[a + j] == y[b +
j]) ? p - 1 : p++;
    }
    for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
        for (k &&k--, j = sa[x[i] - 1]; s[i + k] == s[j + k]; k+
+)
            ;
    }
};

```

5.3. Trie.h

```

class TrieNode {
public:
    unordered_map<char, TrieNode *> children;
    bool isEndOfWord;

    TrieNode() : isEndOfWord(false) {}
};

class Trie {
private:
    TrieNode *root;

public:
    Trie() { root = new TrieNode(); }
    void insert(const string &word) {
        TrieNode *node = root;
        for (char ch : word) {
            if (node->children.find(ch) == node->children.end()) {
                node->children[ch] = new TrieNode();
            }
            node = node->children[ch];
        }
        node->isEndOfWord = true;
    }
    bool search(const string &word) {
        TrieNode *node = root;

```

```

        for (char ch : word) {
            if (node->children.find(ch) == node->children.end()) {
                return false;
            }
            node = node->children[ch];
        }
        return node->isEndOfWord;
    }
    bool startsWith(const string &prefix) {
        TrieNode *node = root;
        for (char ch : prefix) {
            if (node->children.find(ch) == node->children.end()) {
                return false;
            }
            node = node->children[ch];
        }
        return true;
    }
};

```

5.4. hash.h

```

template <int MOD, int P> struct RH {
    // using H1 = RH<1000000007, 91138233>;
    // using H2 = RH<1000000009, 97266353>;
    vector<long long> h, p;
    RH(const string &s) {
        int n = s.size();
        h.resize(n + 1, 0);
        p.resize(n + 1, 0);
        p[0] = 1;
        for (int i = 0; i < n; i++) {
            h[i + 1] = (h[i] * P + s[i]) % MOD;
            p[i + 1] = p[i] * P % MOD;
        }
    }
    long long get(int l, int r) { // [l,r]

```

```

    long long res = (h[r + 1] - h[l] * p[r - l + 1]) % MOD;
    return res < 0 ? res + MOD : res;
}
};

```

5.5. kmp.h

```

vector<int> prefix_function(string s) {
    int n = (int)s.length();
    vector<int> pi(n);
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j])
            j = pi[j - 1];
        if (s[i] == s[j])
            j++;
        pi[i] = j;
    }
    return pi;
}

vector<int> KMP(string text, string pattern) {
    string s = pattern + "#" + text;
    vector<int> pi = prefix_function(s);
    vector<int> matches;
    int p = pattern.length();

    for (int i = 0; i < s.length(); i++) {
        if (pi[i] == p) {
            int match_pos = i - 2 * p;
            matches.push_back(match_pos); // 0-based index in text
        }
    }
    return matches;
}

```

6. Additional

6.1. centroid.h

```

#include <bits/stdc++.h>
void solve() {
    int n;
    cin >> n;
    vvi adj(n);
    int centroid = -1;
    vi sub(n, 0);
    function<void(int, int, int)> find_centroid = [&](int node,
                                                    int par,
                                                    int size) {
        sub[node] = 1;
        for (auto child : adj[node]) {
            if (child == par)
                continue;
            find_centroid(child, node, size);
            sub[node] += sub[child];
        }
        if (centroid == -1) {
            debug(node, sub[node], size);
            if (sub[node] * 2 > size) {
                centroid = node;
            }
        }
    };

    function<void(int, int, int)> getans = [&](int node, int par,
                                              int size) {
        centroid = -1;
        find_centroid(node, -1, size);
        vi v;
        for (auto i : adj[centroid])
            v.pb(i);
        if (sz(v) == 0)

```

```

    ans(centroid);
    int ismin = ask(centroid, v);
    if (ismin)
        ans(centroid);
    if (sz(v) == 1) {
        vvi adjl(n);
        function<void(int, int)> dfscase = [&](int node, int par)
        {
            for (auto child : adj[node]) {
                if (child == par)
                    continue;
                adjl[node].pb(child);
                adjl[child].pb(node);
                dfscase(child, node);
            }
        };
        dfscase(v[0], centroid);
        swap(adj, adjl);
        getans(v[0], -1, sub[v[0]]);
        return;
    }
    function<void(int, int)> getsizes = [&](int node, int par)
    {
        sub[node] = 1;
        for (auto child : adj[node]) {
            if (child == par)
                continue;
            getsizes(child, node);
            sub[node] += sub[child];
        }
    };
    getsizes(centroid, par);
    sort(all(v), [&](int a, int b) { return sub[a] >
sub[b]; });
    vi temp;
    int idx = -1;

```

```

    int totsum = 0;
    for (int i = 0; i < sz(v); i++) {
        totsum += sub[v[i]];
    }
    int sum = 0;
    for (int i = 0; i < sz(v); i++) {
        if (sum >= (totsum / 2)) {
            idx = i;
            break;
        }
        sum += sub[v[i]];
        temp.pb(v[i]);
    }
    ismin = ask(centroid, temp);
    vi bad(n, 0);
    if (ismin) {
        for (auto i : temp)
            bad[i] = 1;
    } else {
        for (auto i : v)
            bad[i] = 1;
        for (auto i : temp)
            bad[i] = 0;
    }
    vvi adjl(n);
    int szz = 0;
    function<void(int, int)> dfs = [&](int node, int par) {
        szz++;
        for (auto child : adj[node]) {
            if (child == par)
                continue;
            if (bad[child] == 1)
                continue;
            adjl[node].pb(child);
            adjl[child].pb(node);
            dfs(child, node);
        }
    };
    dfs(centroid, -1);

```



```

    }
};
dfs(centroid, -1);
swap(adj, adj1);
getans(centroid, -1, sz);
};

getans(0, -1, n);
}

struct LCA {
    int n, LOG;
    vector<vector<int>> up;
    vector<int> depth;
    vector<vector<int>> adj;

    LCA(int n_) {
        n = n_;
        adj.assign(n + 1, {});
        LOG = 1;
        while ((1 << LOG) <= n)
            LOG++;
        up.assign(n + 1, vector<int>(LOG + 1));
        depth.assign(n + 1, 0);
    }

    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    void dfs(int v, int p) {
        up[v][0] = p;
        for (int i = 1; i <= LOG; i++)
            up[v][i] = up[up[v][i - 1]][i - 1];
        for (int to : adj[v])

```

```

            if (to != p) {
                depth[to] = depth[v] + 1;
                dfs(to, v);
            }
    }

    void init(int root) { dfs(root, root); }

    int query(int a, int b) {
        if (depth[a] < depth[b])
            swap(a, b);
        int k = depth[a] - depth[b];
        for (int i = LOG; i >= 0; i--)
            if (k & (1 << i))
                a = up[a][i];
        if (a == b)
            return a;
        for (int i = LOG; i >= 0; i--)
            if (up[a][i] != up[b][i]) {
                a = up[a][i];
                b = up[b][i];
            }
        return up[a][0];
    }

    int get_dist(int u, int v) {
        int lca = query(u, v);
        return depth[u] + depth[v] - 2 * depth[lca];
    }
};

struct CentroidDecomposition {
    vector<vector<int>> adj;
    vector<bool> is_removed;
    vector<int> parent_in_centroid_tree;
    vector<int> subtree_size;
    int n;

```

```
CentroidDecomposition(int size) {
    n = size;
    adj.assign(n + 1, vector<int>());
    is_removed.assign(n + 1, false);
    parent_in_centroid_tree.assign(n + 1, 0);
    subtree_size.assign(n + 1, 0);
}

void add_edge(int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u);
}

int get_subtree_sizes(int u, int p = -1) {
    if (is_removed[u])
        return 0;
    subtree_size[u] = 1;
    for (int v : adj[u]) {
        if (v != p && !is_removed[v]) {
            subtree_size[u] += get_subtree_sizes(v, u);
        }
    }
    return subtree_size[u];
}

int find_centroid(int u, int p, int tree_total_size) {
    for (int v : adj[u]) {
        if (v != p && !is_removed[v] && subtree_size[v] >
tree_total_size / 2) {
            return find_centroid(v, u, tree_total_size);
        }
    }
    return u;
}
```

```
void init(int u = 0, int p = -1) {
    int tree_total_size = get_subtree_sizes(u);
    int centroid = find_centroid(u, -1, tree_total_size);

    parent_in_centroid_tree[centroid] = p;
    is_removed[centroid] = true;

    // --- Add logic for the current centroid here ---

    for (int v : adj[centroid]) {
        if (!is_removed[v]) {
            init(v, centroid);
        }
    }
}

};
```

6.2. dss_sml.h

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
class DSU {
public:
    vector<int> parent;
    vector<int> size;
    void make_set(int v) {
        parent[v] = v;
        size[v] = 1;
    }
    DSU(int n) {
        parent.resize(n);
        size.resize(n);
        for (int i = 0; i < n; i++) {
            make_set(i);
        }
    }
};
```

```

}
int find_set(int v) {
    if (v == parent[v])
        return v;
    return parent[v] = find_set(parent[v]);
}

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) {
        if (size[a] < size[b])
            swap(a, b);
        parent[b] = a;
        size[a] += size[b];
    }
}

};

void solve() {
    int n;
    cin >> n;
    vector<array<int, 2>> edg;
    for (int i = 0; i < n - 1; i++) {
        int a, b;
        cin >> a >> b;
        edg.push_back({a, b});
    }

    vvi adj(n + 1);
    for (int i = 0; i < n - 1; i++) {
        int a, b;
        cin >> a >> b;
        adj[a].pb(b);
        adj[b].pb(a);
    }
}

```

```

DSU dsu(n + 1);
LL ans = 1;

vector<vector<int>> st(n + 1);
for (i, 1, n + 1) st[i].push_back(i);
// debug(st);
for (auto [a, b] : edg) {
    int sz1 = dsu.size[dsu.find_set(a)];
    int sz2 = dsu.size[dsu.find_set(b)];
    ans = mod_mul(ans, mod_mul(inv[sz1], inv[sz2]));

    int pa = dsu.find_set(a);
    int pb = dsu.find_set(b);

    if (st[pa].size() < st[pb].size()) {
        swap(pa, pb);
    }
    // for every edge from comp pb, check if connecting to comp
    pa
    int cnt = 0;
    for (auto node : st[pb]) {
        for (auto j : adj[node]) {
            int par = dsu.find_set(j);
            if (par == pa)
                cnt++;
            if (cnt >= 2)
                break;
        }
    }
    if (cnt != 1) {
        cout << 0 << endl;
        return;
    }

    for (auto node : st[pb]) {

```

```

        st[pa].push_back(node);
    }

    st[pb].clear();

    dsu.union_sets(pa, pb);
}
cout << ans << endl;
}

signed main() {
    fast();
    int t = 1;
    Inverses();
    while (t--)
        solve();
    return 0;
}

```

6.3. xernia_and_tree.h

```

#include <bits/stdc++.h>
using namespace std;

struct HLD {
    ... template <class B> void go_up(int node, int anc, B op) {
        while (top[node] != top[anc]) {
            op(tin[top[node]], tin[node]);
            node = p[top[node]];
        }
        op(tin[anc], tin[node]);
    }
};

template <typename T, typename U> struct seg_tree_lazy {};

struct node {

```

```

    int ans = 1e9, len = 0, max_d = 0;
    node operator+(const node &n) {
        node ret;
        ret.len = len + n.len;
        ret.ans = min(ans + n.len, n.ans);
        ret.max_d = max(max_d, n.max_d);
        return ret;
    }
};

struct update {
    int depth = 1e9;
    node operator()(const node &n) {
        node ret;
        ret.len = n.len;
        ret.ans = min(n.ans, depth - n.max_d);
        ret.max_d = n.max_d;
        return ret;
    }
    update operator+(const update &u) { return {min(depth, u.depth)}; }
};

void solve() {
    int n, m;
    cin >> n >> m;
    vvi adj(n);
    fr(i, 0, n - 1){...}

    vi dep(n, 0);
    function<void(int, int)> dfs = [&](int node, int par) {
        for (auto child : adj[node]) {
            if (child == par)
                continue;
            dep[child] = dep[node] + 1;
            dfs(child, node);
        }
    }
}

```

```

};
dep[0] = 1;
dfs(0, -1);

HLD hld(adj);
seg_tree_lazy<node, update> lst(n);
vector<node> leaves(n);
leaves[hld.tin[0]].ans = 0;
leaves[hld.tin[0]].len = 1;
leaves[hld.tin[0]].max_d = 1;
for (int i = 1; i < n; i++) {
    leaves[hld.tin[i]].ans = 1e9;
    leaves[hld.tin[i]].len = 1;
    leaves[hld.tin[i]].max_d = dep[i];
}
lst.set_leaves(leaves);
while (m--) {
    int q, v;
    cin >> q >> v;
    v--;
    if (q == 1) {
        // paint v red
        int depth = dep[v];
        hld.process(0, v, [&](int l, int r) {
            // in range l, r should do the update
            lst.upd(l, r, {depth});
        });
    } else {
        // get distance from v to closest red vertex
        // should go up?
        auto add = [&](node n2, node n1) {
            // n1 n2
            node ret;
            ret.len = n1.len + n2.len;
            ret.max_d = min(n1.max_d, n2.max_d);
            ret.ans = min(n1.ans, n2.ans + n1.len);
        };
    }
}

```

```

        return ret;
    };
    node ans = {-1, -1, -1};
    hld.process(0, v, [&](int l, int r) {
        // it will keep going up basically
        if (ans.ans == -1) {
            ans = lst.query(l, r);
        } else {
            ans = add(lst.query(l, r), ans);
        }
    });
    cout << ans.ans << endl;
}
}
}

```